



Solution Guide: Lists and Loops

In the previous activity, we added variables to our script and conditional flow control using `if` statements.

Next, you will use loops to automate repetitive tasks in your script.

Loops facilitate code reuse by allowing commands to be run many times without actually typing them repeatedly.

To complete this activity, you will create several `for` loops that satisfy given requirements. If you get to the bonus, you can incorporate a `for` loop into your script.

Instructions

Create your script file.

1. Create a new file named `for_loops.sh` and open it in your text editor.

Solution: `nano for_loops.sh`

2. Add the required `boiler plate` line at the top so that your computer knows it's a Bash script.

Solution: `#!/bin/bash`

Create a `for` loop

Create a `for` loop that scans a list of IP addresses and a list of ports to check. Use 8.8.8.8, 8.8.4.4, and 1.1.1.1 for the IP addresses. Use ports 80 and 443 for port check.

Solution:

Unset

- `#!/bin/bash`
-

```

○ # List of hosts to check
○ hosts=(
○     "8.8.8.8"
○     "8.8.4.4"
○     "1.1.1.1"
○ )
○
○ # List of ports to check
○ ports=(
○     "80"
○     "443"
○ )
○
○ # Loop through each host
○ for host in "${hosts[@]}"; do
○     echo "Checking connectivity to host: $host"
○
○     # Check if host is reachable
○     if ping -c 1 "$host" &> /dev/null; then
○         echo "Host $host is reachable."
○
○         # Nested loop to check each port on the current
host
○         for port in "${ports[@]}"; do
○             if nc -z -w 1 "$host" "$port" &> /dev/null; then
○                 echo "Port $port is open on host $host."
○             else
○                 echo "Port $port is NOT open on host $host."
○             fi
○         done
○     else
○         echo "Host $host is NOT reachable."
○     fi
○
○     # Add a blank line for readability
○     echo

```

- done

Super Bonus

1. During the last exercise, you created a variable that holds the command `find / -type f -perm /4000 2> /dev/null` and then you used `echo` to print out your variable later in the script.

You may have noticed that this produces an output that is a bit jumbled together:

Unset

- Exec Files:
- /home/sysadmin/Documents/setup_scripts/sysadmin/day3_stu_setup.sh
- /home/instructor/Documents/setup_scripts/sysadmin/day3_stu_setup.sh
- /home/instructor/Documents/setup_scripts/instructor/day3_setup.sh
-

Challenge

Instead of using `echo` to print out this variable, use a `for` loop to print out each file on its own line.

Solution:

Unset

- `execs=$(find /home -type f -perm 777 2> /dev/null)`
-
- `for exec in ${execs[@]}`
- `do`
- `echo $exec`
- `done`

